In [2]:
```
from sklearn.datasets import load_iris
from  sklearn.model_selection  import(train_test_split,cross_val_score,Grid
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from  sklearn.metrics  import(accuracy_score,classification_report,confusio
```

In [3]:
```
#Load the dataset
iris=load_iris()
X=iris.data
y=iris.target
```

In [4]:
```
#Split the dataset
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_s
```

In [5]:
```
#Create a Pipeline
pipeline = Pipeline([
    ("scaler",StandardScaler()),
    ("classifier",LogisticRegression(max_iter=1000))])
```

In [6]:
```
#Analyze baseline performance using cross- validation
cv_scores=  cross_val_score(pipeline,X_train,y_train,cv=5,scoring="accurac

print("Cross-validation Scores:", cv_scores)
print("Mean CV Accuracy:", round(cv_scores.mean(), 4))
```

```
Cross-validation Scores: [0.91666667 0.95833333 0.95833333 0.95833333 1.
]
Mean CV Accuracy: 0.9583
```

In [7]:
```
# Train the baseline model
pipeline.fit(X_train,y_train)
baseline_pred= pipeline.predict(X_test)

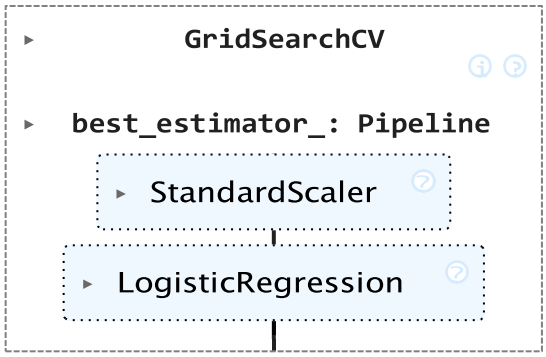
baseline_accuracy = accuracy_score(y_test, baseline_pred)
print("\nBaseline Test Accuracy:", round(baseline_accuracy*100,2),"%")
```

```
Baseline Test Accuracy: 93.33 %
```

In [9]:
```
#Optimize hyperparameters
param_grid = {"classifier_C": [0.01,0.1,1,10,100]}

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1)
grid_search.fit(X_train,y_train)
```

Out[9]:



In [10...

```
#Get the best model
best_model = grid_search.best_estimator_

print("\nBest Parameters:", grid_search.best_params_)
print("Best CV Accuracy:" , round(grid_search.best_score_, 4))
```

```
Best Parameters: {'classifier__C': 10}
Best CV Accuracy: 0.9667
```

In [11...

```
#Evaluate the optimized model
optimized_pred = best_model.predict(X_test)

optimized_accuracy = accuracy_score(y_test, optimized_pred)

print("\nOptimized Test Accuracy:", round(optimized_accuracy*100,2),"%")

#print("Confusion Matrix:")
#print(confusion_matrix(y_test,optimized_pred))
```

```
Optimized Test Accuracy: 100.0 %
```

In [12...

```
#Compare the models
print("\nBaseline  Accuracy:",round(baseline_accuracy*100,2),  "%")

print("\nOptimized  Accuarcy:",round(optimized_accuracy*100,2),"%")
```

```
Baseline Accuracy: 93.33 %

Optimized  Accuarcy: 100.0 %
```